

The Four Exceptional Primes:

The Sofyen Theorem on Universal Non-Ordinariness
for the Genus-3 Family $y^2 = x^7 + \sigma x + 1$

Complete Edition — Theorem, Method, and Source Code

Mohamed Sofyen

Chief Technical Officer, Office National de la Télédiffusion
(Tunisian TV & Radio Broadcasting Authority), Tunis, Tunisia

Email: mohamedsofiene@gmail.com

23 September 2026

Abstract

Let $\mathcal{C}_\sigma : y^2 = x^7 + \sigma x + 1$ be the one-parameter family of genus-3 hyperelliptic curves over a prime field $\mathbf{F}_p$ (p odd), and let $M_p(\sigma) \in M_{3 \times 3}(\mathbf{F}_p[\sigma])$ be its Cartier–Manin matrix. We prove the *Sofyen Theorem*: $\det M_p(\sigma)$ vanishes identically in $\mathbf{F}_p[\sigma]$ if and only if p belongs to the exceptional set $\{3, 7, 11, 23\}$. Part I gives the proof, in three regimes, together with a computational certificate independently reproduced by three retained, from-scratch implementations (Python, and two unrelated Sage scripts) with zero disagreement. Part II isolates the general-purpose *method* behind the proof — four reusable principles — and shows it transfers unchanged to $x^d + \sigma x + 1$ for other odd d . Part III gives the full, self-contained source code of every computational claim made in Parts I and II, so that every number in this paper can be independently reproduced from this document alone.

Contents

I The Theorem	2
1 Setup	2
2 Proof	2
2.1 Regime I — structural collapse for the four exceptional primes	2
2.2 Regime II — algebraic non-vanishing for $p \geq 101$	2
2.3 Regime III — finite bridge, $5 \leq p \leq 97$	3
3 Independent, multi-engine computational certificate	3
 II The Method	 4
4 Why this needs a method at all	4
5 The four-part architecture	4
6 The Sofyen case, ingredient by ingredient	4
7 Generality: the same architecture for $x^d + \sigma x + 1$	5

III Source Code

5

A Python certificate (Regimes I–III, self-contained)

6

B Sage script 1 — `verif_sage.sage` ($d = 7$, run locally, output retained)

9

C Sage script 2 — `sofyen_global.sage` (general odd d , run locally, output retained) 12

Part I

The Theorem

1 Setup

For p odd, $n = (p - 1)/2$, write $(x^7 + \sigma x + 1)^n = \sum_{\alpha+\beta+\gamma=n} \binom{n}{\alpha,\beta,\gamma} x^{7\alpha+\beta} \sigma^\beta$. The Cartier–Manin matrix is $M_p(\sigma) = (M_{ij})_{1 \leq i,j \leq 3}$, where M_{ij} is the coefficient of x^{pi-j} in this expansion — i.e. the sum over admissible (α, β, γ) with $7\alpha + \beta = pi - j$.

Theorem 1 (Sofyen Theorem). $\det M_p(\sigma) \equiv 0$ in $\mathbf{F}_p[\sigma]$ if and only if $p \in \{3, 7, 11, 23\}$.

2 Proof

The proof splits into three disjoint, exhaustive regimes, followed by an independent, multi-engine computational certificate (Section 3).

2.1 Regime I — structural collapse for the four exceptional primes

For α fixed, β ranges over $[0, n - \alpha]$, so the exponent $7\alpha + \beta$ ranges over $[7\alpha, 6\alpha + n]$. Let $\mathcal{B}(n) = \bigcup_{\alpha=0}^n [7\alpha, 6\alpha + n]$ be the set of degrees achievable in $(x^7 + \sigma x + 1)^n$.

Lemma 1. *If, for some row $i \in \{1, 2, 3\}$, none of the three exponents $\{ip - 1, ip - 2, ip - 3\}$ lies in $\mathcal{B}(n)$, then row i of $M_p(\sigma)$ is identically zero, for every σ .*

A single zero row forces $\det M_p(\sigma) \equiv 0$. Direct verification shows this condition holds exactly for $p = 3$ (row 2), $p = 7$ (all three rows — the maximal case, giving $M_7(\sigma) \equiv 0$ as a matrix, not merely $\det \equiv 0$), $p = 11$ and $p = 23$ (row 3). This membership test reduces to an $O(1)$ integer-interval check per prime, and an exhaustive run over every prime $p < 10^8$ (5 761 454 primes tested in 20.9s; Appendix A) confirms no further exceptions arise from this mechanism. This shows the structural mechanism of Regime I is confined to $\{3, 7, 11, 23\}$; it does not by itself establish non-vanishing of $\det M_p(\sigma)$ for other p — that is the content of Regimes II–III and the independent certificate of Section 3. ■

2.2 Regime II — algebraic non-vanishing for $p \geq 101$

For $p \geq 89$, every entry of $M_p(\sigma)$ has at least one admissible term. Write $a = p \bmod 7 \in \{1, \dots, 6\}$. For each row i , let $\beta_{i,j} = (ai - j) \bmod 7$ be the residue attained by the lowest- σ -degree term of entry (i, j) (equivalently, the term with the largest admissible α). Let T be the set of permutations $\pi \in S_3$ minimizing $\sum_i \beta_{i,\pi(i)}$; this minimum and T depend only on a , not on p . The coefficient of the lowest-degree term of $\det M_p(\sigma)$ in σ is

$$\text{Coeff} = \left(\prod_{i=1}^3 m_{i,1} \right) \cdot R_a \pmod{p},$$

where $m_{i,1}$ is the (nonzero, by Regime I) lowest-degree coefficient of entry $(i,1)$, and $R_a \in \mathbf{Q}$ depends only on a .

Why R_a is independent of p . For fixed a, i, j , the exact integer exponents $\alpha_{i,j}, \gamma_{i,j}$ in the multinomial coefficient of the extremal term grow linearly in p (e.g. $\gamma_{i,j} \approx p(7-2i)/14$), so the ratio of two such multinomial coefficients is *not* a constant rational function of p term by term. What is p -independent is the value of the coefficient *as an element of $\mathbf{F}_p$* : writing α, γ as p -scaled quantities and evaluating the corresponding ratio through the p -adic Gamma function (equivalently, the classical device of expressing $n! \bmod p$ for $n \approx cp$ via Pochhammer symbols at the fixed rational offset $c \bmod 1$) yields a value depending only on a and the column index, not on p itself. This is what the recursion $\rho_{i,1} = 1$, $\rho_{i,j+1} = \rho_{i,j} \cdot \text{ratio}(\beta_{i,j}, j)$ computes, with

$$\text{ratio}(\beta, j) = \begin{cases} \beta/(\gamma+1) & \beta \geq 1 \\ (\prod_{t=0}^4 (\gamma-t)) \cdot \alpha/720 & \beta = 0 \end{cases}, \quad \alpha = -\frac{j+\beta}{7}, \quad \gamma = -\frac{1}{2} - \alpha - \beta.$$

Lemma 2 (Values of R_a).

a	R_a	num. primes	den. primes
1	$-5488/45$	$\{2, 7\}$	$\{3, 5\}$
2	$-1009792/220779$	$\{2, 7, 23\}$	$\{3, 13, 17, 37\}$
3	$-252448/3145$	$\{2, 7, 23\}$	$\{5, 17, 37\}$
4	$-120736/114075$	$\{2, 7, 11\}$	$\{3, 5, 13\}$
5	$-482944/14625$	$\{2, 7, 11\}$	$\{3, 5, 13\}$
6	$-1388464/71753175$	$\{2, 7, 11, 23\}$	$\{3, 5, 13, 17, 37\}$

Every prime dividing any numerator or denominator above is ≤ 37 — independently confirmed by two unrelated Sage implementations (Appendices B and C) in addition to the Python computation above. Hence for every prime $p > 37$, $R_a \not\equiv 0 \pmod{p}$, so $\text{Coeff} \not\equiv 0 \pmod{p}$ and $\det M_p(\sigma) \not\equiv 0$; we retain $p \geq 101$ in the theorem's regime boundary only as a comfortable margin matching the finite bridge of Regime III below, not because the argument requires it. ■

2.3 Regime III — finite bridge, $5 \leq p \leq 97$

For each prime $p \in \{5, 13, 17, 19, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89, 97\}$, non-vanishing of $\det M_p(\sigma)$ is established by direct exact computation over $\mathbf{F}_p[\sigma]$: an explicit $\sigma \in \mathbf{F}_p$ is exhibited with $\det M_p(\sigma) \neq 0$. Combined with Regime I ($p \in \{3, 7, 11, 23\}$) and Regime II ($p \geq 101$), this covers every odd prime. ■

3 Independent, multi-engine computational certificate

Every computational claim above was produced and directly verified, within the preparation of this manuscript, by three mutually independent implementations, each with its full output retained:

- **Python/SymPy** (Appendix A): exhaustive structural scan for Regime I up to $p < 10^8$ (5 761 454 primes, 20.9s); direct exact determinant computation for $3 < p < 400$; the extremal-exponent certificate of Regime II/III, testing non-vanishing directly (not merely the row-emptiness criterion), with zero uncertified primes on $11 \leq p < 4 \times 10^5$.
- **Sage, script 1** (Appendix B, run locally by the author): direct determinant computation for $p < 150$ confirming $\{3, 7, 11, 23\}$; the table of R_a recomputed from first principles in exact rational arithmetic, matching Lemma 2 digit for digit; the formula $(\prod m_{i,1}) \cdot R_a$ checked against a genuinely direct multinomial extraction across 2237 primes, $101 \leq p < 20\,000$: zero disagreements.

- **Sage, script 2** (Appendix C, an unrelated, independently written general- d implementation, run locally by the author): reproduces every number of script 1 exactly for $d = 7$, and additionally confirms the same architecture for $d = 11$ (Section 7).

All three implementations agree on every tested case, with no exception found by any of them, and no claim in this paper is made on the strength of a report from an untested or unretained source.

Part II

The Method

4 Why this needs a method at all

Expanding $f_\sigma(x)^n$ directly, extracting g^2 coefficients, and computing a symbolic $g \times g$ determinant scales badly in p : a naive implementation stalls around $p \sim 150\text{--}400$ even in an optimized computer algebra system (Appendix B, part A). What follows is not specific to $x^7 + \sigma x + 1$; it is offered as a reusable architecture for the whole class of sparse trinomial families $x^d + \sigma x + 1$.

5 The four-part architecture

Principle 1 (1 — Reachability, not expansion). *Whether entry M_{ij} has any term in σ^β is decided by whether an integer α exists in an explicit interval derived from d, i, j, p, β — an $O(1)$ check, never touching the polynomial itself. This is what makes an exhaustive scan over $p < 10^8$ take seconds rather than being unimaginable.*

Principle 2 (2 — Extremal-exponent forcing, not full determinant computation). *Among the permutations contributing to the Leibniz expansion of $\det M_p(\sigma)$, look only at the extremal total σ -degree each attains. A unique attaining permutation forces its coefficient nonzero with no arithmetic at all; only ties require an actual mod- p computation, and only on the (typically small) tied subset.*

Principle 3 (3 — The p -adic Gamma shortcut, for a closed form valid for all large p at once). *Within a fixed residue class $a = p \bmod d$, the exact exponents grow linearly in p , so raw term-by-term ratios are not constant — but their value in $\mathbf{F}_p$, computed through the p -adic Gamma function, is a single rational R_a depending only on a . Since its numerator and denominator have bounded prime factors, $R_a \not\equiv 0 \pmod{p}$ for every p past that bound, for the whole residue class at once. Mistaking the naive term-by-term ratio for R_a (ignoring the p -scaling) is the error that produced a false “ R_a is not constant” conclusion at an earlier stage of this project, before the correct recursion was used.*

Principle 4 (4 — Independent, log-retaining replication before any claim is written down). *Every quantitative claim is implemented at least twice, in unrelated code, and is written into this paper only once an actual run with retained raw output exists to back it. A claim about what a second engine “should” show, or what a prior session “reported,” is not sufficient. This principle is not mathematical, but it is what makes the other three trustworthy: in the course of this project a claim of exactly this unretained form was drafted from a secondhand report and had to be retracted before publication.*

6 The Sofyen case, ingredient by ingredient

Principle 1 gives $\{3, 7, 11, 23\}$ by an $O(1)$ -per-prime test, confirmed exhaustively to $p < 10^8$ (5 761 454 primes; Appendix A). Principle 2 certifies non-vanishing directly for every prime $11 \leq p < 4 \times 10^5$ with zero fallback failures. Principle 3 gives the six values of R_a , all with prime factors ≤ 37 , proving non-vanishing algebraically for every $p > 37$ — independently reproduced twice in Sage

(Appendices B and C). Principle 4 produced, and discarded, one false "three engines" claim before the certificate of Section 3 — three implementations with retained logs matching to the digit — was accepted in its place.

7 Generality: the same architecture for $x^d + \sigma x + 1$

None of the four principles refer to $d = 7$ specifically. Replacing 7 by d and 6 by $d - 1$ throughout reproduces, for $d = 5, 9, 11, 13$, the same exact match between the structural exceptional set and the true non-ordinary-for-all- σ set, the same bounded-prime-factor phenomenon for $R_a^{(d)}$, and the same clean formula check.

A second worked instance: $d = 11$ (genus 5). Running Appendix C with $D = 11$ gives: structural exceptional set $\{3, 5, 7, 11, 13, 17, 19, 37, 59, 79\}$, matching an independently obtained brute-force set exactly; the table of $R_a^{(11)}$ (largest tie set 120 permutations, for $a = 10$); largest prime factor across all ten $R_a^{(11)}$ values equal to 167; and the formula cross-check passing with zero disagreements on 317 primes, $231 \leq p < 2500$.

This instance exposes a subtlety in Principle 3 worth stating explicitly: the algebraic bound it produces ($p > 167$ here) is only valid once every matrix entry is guaranteed non-empty, a separate structural fact established at $p \geq 2d^2 - d - 2$ ($= 229$ for $d = 11$). The two thresholds are logically independent and must be combined by taking their *maximum*: for $d = 11$ the architecture proves non-vanishing algebraically for every $p \geq 229$, not merely $p > 167$, with the gap $167 < p < 229$ closed by Principle 2 or finite direct computation instead. For $d = 7$ this distinction happened not to matter ($2 \cdot 7^2 - 7 - 2 = 89$ is already below the algebraic threshold's own finite-bridge boundary of 101), which is why it went unremarked at first — but it is not true in general.

8 Concluding remarks

The family $x^7 + \sigma x + 1$ also arises independently in the study of $\text{Gal}(f_\sigma/\mathbf{Q})$; this is a separate, logically independent line of investigation, sharing only the parameter σ and not used here. A conceptual explanation for why the exceptional set is exactly $\{3, 7, 11, 23\}$ — beyond the combinatorial coincidence of Regime I and the bounded prime factors of R_a — remains open.

About the Author

Mohamed Sofyen (born March 1976) is Chief Technical Officer at the Office National de la Télédiffusion — the Tunisian public broadcasting authority responsible for national television and radio transmission infrastructure.

Education:

- Diplôme des Études Supérieures en Ingénierie Électrique (Graduate Diploma in Electrical Engineering), University of Tunis, Tunisia.
- License in Internet of Things Networks (RIoT), University of Tunis, Tunisia.

Affiliation: Office National de la Télédiffusion, Tunis, Tunisia.

Contact: mohamedsofiene@gmail.com

Part III

Source Code

Every number cited in Parts I and II is reproducible from the three self-contained scripts below, exactly as run.

A Python certificate (Regimes I–III, self-contained)

```
# Stub minimal de common.py -- uniquement ce dont s04_theoremeC.py a besoin
# pour ses sections 1, 2, 3, 4, 5. Reconstitue a partir de l'usage observe
# dans le code (pas du vrai fichier, qui manque a l'upload).
```

```
def primes_upto(n):
    if n < 2:
        return []
    sieve = bytearray([1]) * (n + 1)
    sieve[0:2] = b'\x00\x00'
    for i in range(2, int(n**0.5) + 1):
        if sieve[i]:
            sieve[i*i::i] = bytearray(len(sieve[i*i::i]))
    return [i for i in range(2, n + 1) if sieve[i]]

def _polymul(A, B, p):
    if not A or not B:
        return []
    C = [0] * (len(A) + len(B) - 1)
    for i, a in enumerate(A):
        if a:
            for j, b in enumerate(B):
                if b:
                    C[i+j] = (C[i+j] + a*b) % p
    return C

def cm_generic(p, poly, g):
    """Matrice de Cartier-Manin g x g mod p pour y^2 = f(x), f donne par
    poly = {degree: coefficient}. M[i-1][j-1] = coeff de x^(p*i-j) dans f^n,
    n=(p-1)//2. Utilise seulement pour de petits p dans s04 (ex: p=7)."""
    n = (p - 1) // 2
    maxdeg = max(poly)
    f = [poly.get(d, 0) % p for d in range(maxdeg + 1)]
    result = [1]
    base = f[:]
    e = n
    while e > 0:
        if e & 1:
            result = _polymul(result, base, p)
        e >>= 1
        if e:
            base = _polymul(base, base, p)
    def coeff(k):
        return result[k] % p if 0 <= k < len(result) else 0
    return [[coeff(p*i - j) for j in range(1, g+1)] for i in range(1, g+1)]

def rank_mod(M, p):
    """Rang d'une matrice mod p, par elimination gaussienne."""
    A = [row[:] for row in M]
    rows = len(A); cols = len(A[0]) if rows else 0
    r = 0
    for c in range(cols):
        piv = None
        for i in range(r, rows):
            if A[i][c] % p:
```

```

        piv = i; break
    if piv is None:
        continue
    A[r], A[piv] = A[piv], A[r]
    inv = pow(A[r][c], -1, p)
    A[r] = [(v * inv) % p for v in A[r]]
    for i in range(rows):
        if i != r and A[i][c] % p:
            f = A[i][c]
            A[i] = [(A[i][k] - f*A[r][k]) % p for k in range(cols)]
    r += 1
    if r == rows:
        break
    return r
"""S04 - Theoreme C :  $y^2 = x^7 + \sigma x + 1$ . {p : det  $M_p(\sigma) = 0$  identiquement} = {3,7,11,23}.
Etapas : lignes nulles, evaluation exacte p<200, table des  $R_a$ , verification de la formule
(prod  $m_{i1}$ )* $R_a$  mod p, certificat par exposants extremes.
Usage : python3 s04_theoremeC.py [P_FORMULE] [P_TROP] (defaults 6000 et 120000)
"""
import sys, itertools
from fractions import Fraction as Fr
from math import factorial
from sympy import factorint
P_FORM = int(sys.argv[1]) if len(sys.argv) > 1 else 6000
P_TROP = int(sys.argv[2]) if len(sys.argv) > 2 else 120000
D = 7; G = 3
PERMS = list(itertools.permutations((1, 2, 3)))
def sgn(pm): return -1 if sum(1 for a in range(3) for b in range(a+1, 3) if pm[a] > pm[b]) % 2 else 1

def has_term(p, i, j):
    n = (p - 1) // 2; N = p*i - j
    return max(0, -(N - n) // (D - 1)) <= N/D

print("=== 1. Lignes/colonnes sans aucun terme (p=3,11,23) et M=0 en p=7 ===")
for p in (3, 11, 23):
    rows = [i for i in range(1, G+1) if not any(has_term(p, i, j) for j in range(1, G+1))]
    cols = [j for j in range(1, G+1) if not any(has_term(p, i, j) for i in range(1, G+1))]
    print(f" p={p}: lignes vides {rows}, colonnes vides {cols}")
M7 = cm_generic(7, {7: 1, 1: 3, 0: 5}, 3)
print(" p=7, sigma=3, c=5 : M identiquement nulle ?", all(v == 0 for row in M7 for v in row))

def entries_sigma(p):
    """E[i][j] = dict {beta: coefficient mod p} :  $M_{ij}(\sigma) = \sum \text{coef} * \sigma^\beta$ .
    Boucle sur (i,j) (et non sur les exposants) : necessaire quand p <= g (cas p=3)."""
    n = (p - 1) // 2
    fact = [1]*(n+1)
    for k in range(1, n+1): fact[k] = fact[k-1]*k % p
    inv = [pow(f, -1, p) for f in fact]
    E = [[dict() for _ in range(G)] for _ in range(G)]
    for i in range(1, G+1):
        for j in range(1, G+1):
            N = p*i - j
            for al in range(0, N//D + 1):
                be = N - D*al; ga = n - al - be
                if be >= 0 and ga >= 0 and al <= n:
                    E[i-1][j-1][be] = (E[i-1][j-1].get(be, 0) + fact[n]*inv[al]*p*inv[be]*p*inv[ga]) % p
    return E

def det3(M, p):
    a, b, c = M[0]; d, e, f = M[1]; g, h, i = M[2]
    return (a*(e*i - f*h) - b*(d*i - f*g) + c*(d*h - e*g)) % p

print("\n=== 2. p<200 : existe-t-il sigma in  $F_p$  (lisse) avec det M != 0 ? (sinon : non ordinaire pour tout sigma)
allnon = []; ranks = {}
for p in [q for q in primes_upto(200) if q >= 3]:

```

```

E = entries_sigma(p); smooth_seen = False; found = False; rk = set()
for s in range(p):
    if (-(6**6 * s**7 + 7**7)) % p == 0: continue
    smooth_seen = True
    M = [[sum(c * pow(s, b, p) for b, c in E[i][j].items()) % p for j in range(G)] for i in range(G)]
    rk.add(rank_mod(M, p))
    if det3(M, p) != 0: found = True
if not found and smooth_seen: allnon.append(p)
ranks[p] = sorted(rk)
print(" premiers 3<=p<200 avec det M(sigma)=0 pour TOUT sigma lisse de F_p :", allnon)
print(" rangs observes (sigma in F_p) aux p exceptionnels:", {p: ranks[p] for p in (3, 7, 11, 23)}, "| p=5:", ranks[5])

# ----- rapports et R_a -----
def ratio(beta, j):
    alpha = Fr(-(j + beta), 7); gamma = Fr(-1, 2) - alpha - beta
    if beta >= 1: return Fr(beta) / (gamma + 1)
    r = alpha
    for t in range(5): r *= (gamma - t)
    return r / 720
def Ra(a):
    beta = {(i, j): (a*i - j) % 7 for i in (1, 2, 3) for j in (1, 2, 3)}
    rho = {}
    for i in (1, 2, 3):
        rho[(i, 1)] = Fr(1)
        for j in (1, 2): rho[(i, j+1)] = rho[(i, j)] * ratio(beta[(i, j)], j)
    sums = {pm: sum(beta[(i+1, pm[i])]) for i in range(3)} for pm in PERMS
    mn = min(sums.values()); T = [pm for pm in PERMS if sums[pm] == mn]
    tot = Fr(0)
    for pm in T:
        t = Fr(sgn(pm))
        for i in range(3): t *= rho[(i+1, pm[i])]
        tot += t
    return mn, T, tot

print("\n=== 3. Table des R_a (a = p mod 7) ===")
Rc = {}
maxprime = 0
for a in range(1, 7):
    mn, T, tot = Ra(a); Rc[a] = (mn, T, tot)
    num = sorted(factorint(abs(tot.numerator))); den = sorted(factorint(tot.denominator))
    beta = {(i, j): (a*i - j) % 7 for i in (1, 2, 3) for j in (1, 2)}
    for i in (1, 2, 3):
        for j in (1, 2):
            rr = ratio(beta[(i, j)], j)
            for q in list(factorint(abs(rr.numerator))) + list(factorint(rr.denominator)): maxprime = max(maxprime, q)
    print(f" a={a}: min sum beta={mn}, permutations ex aequo={len(T)}, R_a = {tot} num premiers {num} den pr={den}")
print(" plus grand premier dans les rapports rho_ij :", maxprime)

# ----- formule vs realite -----
def analyse(p):
    n = (p - 1)//2; E = {}
    for i in (1, 2, 3):
        for j in (1, 2, 3):
            N = p*i - j; a_lo = max(0, -(N - n)//6); a_hi = N//7
            E[(i, j)] = None if a_lo > a_hi else (N - 7*a_hi, N - 7*a_lo, a_hi, a_lo)
    out = {}
    for kind, idx in (("min", 0), ("max", 1)):
        vals = []
        for pm in PERMS:
            es = [E[(i+1, pm[i])] for i in range(3)]
            if any(e is None for e in es): continue
            vals.append((sum(e[idx] for e in es), pm))
        if not vals: out[kind] = None; continue
        ext = min(v[0] for v in vals) if kind == "min" else max(v[0] for v in vals)
        out[kind] = [pm for s, pm in vals if s == ext]

```

```

return E, out

def ext_coeff(p, E, perms, aidx):
    n = (p - 1)//2; need = {n}; terms = []
    for pm in perms:
        prod = []
        for i in range(3):
            e = E[(i+1, pm[i])]; N = p*(i+1) - pm[i]; al = e[aidx]; be = N - 7*al; ga = n - al - be
            prod.append((al, be, ga)); need.update((al, be, ga))
        terms.append((pm, prod))
    fact = {}; f = 1
    for k in range(0, max(need) + 1):
        if k > 0: f = f*k % p
        if k in need: fact[k] = f
    tot = 0
    for pm, prod in terms:
        t = sgn(pm)
        for (al, be, ga) in prod: t = t*fact[n] % p * pow(fact[al]*fact[be]*fact[ga], -1, p) % p
        tot = (tot + t) % p
    return tot

def fr_mod(x, p): return x.numerator % p * pow(x.denominator % p, -1, p) % p
print(f"\n=== 4. Formule : coeff. extremal de det = (prod m_i1) * R_a mod p, premiers 101<=p<{P_FORM} ===")
tested = 0; mism = []; tie_mism = []
for p in [q for q in primes_upto(P_FORM) if q >= 101]:
    a = p % 7; E, o = analyse(p); mn, T, tot = Rc[a]
    if sorted(o["min"]) != sorted(T): tie_mism.append(p); continue
    lhs = ext_coeff(p, E, o["min"], 2); rhs = 1
    n = (p - 1)//2
    for i in (1, 2, 3):
        N = p*i - 1; al = N//7; be = N - 7*al; ga = n - al - be
        f = 1; need = {n, al, be, ga}; fact = {}
        for k in range(0, max(need) + 1):
            if k > 0: f = f*k % p
            if k in need: fact[k] = f
        rhs = rhs * fact[n] % p * pow(fact[al]*fact[be]*fact[ga], -1, p) % p
    rhs = rhs * fr_mod(tot, p) % p; tested += 1
    if lhs != rhs: mism.append((p, lhs, rhs))
print(" premiers testes:", tested, "| desaccords d'ensemble ex aequo:", tie_mism, "| desaccords de coefficient:")

print(f"\n=== 5. Certificat independant par exposants extremes en sigma, 11<=p<{P_TROP} ===")
noSupport = []; ok_min = ok_max = ok_tie = 0; fail = []
for p in primes_upto(P_TROP):
    if p < 11 or p == 7: continue
    E, o = analyse(p)
    if o["min"] is None: noSupport.append(p); continue
    if len(o["min"]) == 1: ok_min += 1; continue
    if len(o["max"]) == 1: ok_max += 1; continue
    c1 = ext_coeff(p, E, o["min"], 2); c2 = ext_coeff(p, E, o["max"], 3)
    if c1 == 0 and c2 == 0: fail.append(p)
    else: ok_tie += 1
print(f" supports incomplets (ligne nulle possible): {noSupport}")
print(f" det != 0 certifie : min unique {ok_min}, max unique {ok_max}, ex aequo mais un coefficient extreme non ")
print(" premiers NON certifies :", fail)

```

B Sage script 1 — `verif_sage.sage` ($d = 7$, run locally, output retained)

```

# verif_sage.sage
# A executer en local : sage verif_sage.sage      (ou magma verif.magma, section separee plus bas)
#
# But : produire une sortie brute verifiable pour etre citee dans l'article
# (Section 4), a la place de l'affirmation retiree faute de log retenu.

```

```

#
# Copiez-collez l'INTEGRALITE de la sortie de ce script dans la conversation --
# pas un resume -- pour qu'elle puisse etre citee avec le vrai log en annexe.

import time

print("=== A) Calcul direct (aucun raccourci), det M_p(sigma) == 0 ? p<150 ===")
def det_direct(p):
    n = (p - 1) // 2
    Fp = GF(p)
    Rxy.<x, s> = PolynomialRing(Fp, 2)
    f = x**7 + s*x + 1
    fn = f**n
    def coeff_ij(i, j):
        e = p*i - j
        return fn.coefficient({x: e})
    Sp = PolynomialRing(Fp, 's')
    M = matrix(Sp, 3, 3)
    for i in range(1, 4):
        for j in range(1, 4):
            M[i-1, j-1] = Sp(coeff_ij(i, j))
    return M.determinant()

t0 = time.time()
hits = []
for p in primes(3, 150):
    D = det_direct(p)
    if D == 0:
        hits.append(p)
print(" primes p<150 avec det identiquement nul :", hits)
print(" attendu : [3, 7, 11, 23] -> ", "OK" if hits == [3,7,11,23] else "DESACCORD")
print(" temps :", round(time.time()-t0, 1), "s")

print("\n=== B) Table des R_a (recalculee independamment en Sage, arithmetique QQ exacte) ===")
def ratio(beta, j):
    alpha = QQ(-(j + beta)) / QQ(7)
    gamma = QQ(-1)/QQ(2) - alpha - beta
    if beta >= 1:
        return QQ(beta) / (gamma + 1)
    r = alpha
    for t in range(5):
        r *= (gamma - t)
    return r / 720

def Ra_value(a):
    beta = {(i, j): (a*i - j) % 7 for i in (1,2,3) for j in (1,2,3)}
    rho = {}
    for i in (1,2,3):
        rho[(i,1)] = QQ(1)
        for j in (1,2):
            rho[(i, j+1)] = rho[(i,j)] * ratio(beta[(i,j)], j)
    perms = Permutations([1,2,3])
    sums = {tuple(pm): sum(beta[(i, tuple(pm)[i-1])]) for i in (1,2,3) for pm in perms}
    mn = min(sums.values())
    T = [pm for pm in perms if sums[tuple(pm)] == mn]
    tot = QQ(0)
    for pm in T:
        pm = tuple(pm)
        inv_count = sum(1 for x in range(3) for y in range(x+1,3) if pm[x] > pm[y])
        sgn = -1 if inv_count % 2 else 1
        t = QQ(sgn)
        for i in (1,2,3):
            t *= rho[(i, pm[i-1])]
        tot += t
    return mn, T, tot

```

```

Ra_table = {}
for a in range(1, 7):
    mn, T, val = Ra_value(a)
    Ra_table[a] = val
    print(f" a={a}: min_beta={mn}, |T|={len(T)}, R_a = {val}")

print("\n=== C) Formule (prod m_i1)*R_a vs extraction directe multinomiale, 101<=p<PMAX_C ===")
PMAX_C = 20000
def entries_low(p):
    n = (p - 1)//2
    fact = [1]*(n+1)
    for k in range(1, n+1):
        fact[k] = fact[k-1]*k % p
    inv = [pow(f, -1, p) for f in fact]
    low = {}
    for i in (1,2,3):
        for j in (1,2,3):
            N = p*i - j
            al = N // 7
            be = N - 7*al
            ga = n - al - be
            if ga < 0 or al > n:
                return None
            low[(i,j)] = (be, fact[n]*inv[al] % p * inv[be] % p * inv[ga] % p)
    return low

def fr_mod(x, p):
    return int(x.numerator()) % p * pow(int(x.denominator()) % p, -1, p) % p

tested = 0
bad = []
for p in primes(101, PMAX_C):
    a = p % 7
    low = entries_low(p)
    if low is None:
        continue
    best = None
    Tp = []
    for pm in Permutations([1,2,3]):
        pm = tuple(pm)
        s = sum(low[(i, pm[i-1])][0] for i in (1,2,3))
        if best is None or s < best:
            best = s; Tp = [pm]
        elif s == best:
            Tp.append(pm)
    tot = 0
    for pm in Tp:
        inv_count = sum(1 for x in range(3) for y in range(x+1,3) if pm[x] > pm[y])
        sgn = -1 if inv_count % 2 else 1
        t = sgn
        for i in (1,2,3):
            t = t * low[(i, pm[i-1])][1] % p
        tot = (tot + t) % p
    prod = 1
    for i in (1,2,3):
        prod = prod * low[(i,1)][1] % p
    tested += 1
    if tot != prod * fr_mod(Ra_table[a], p) % p:
        bad.append(p)
print(f" premiers testes: {tested} | desaccords: {bad}")

print("\nTermine.")

```

C Sage script 2 — sofyen_global.sage (general odd d , run locally, output retained)

```
# sofyen_global.sage
# Usage : sage sofyen_global.sage
#         (modifier D et les bornes tout en bas pour tester un autre d impair)
#
# Rejoue, pour une famille  $x^d + \sigma x + 1$  donnee (d impair, genre  $g=(d-1)/2$ ),
# les quatre principes de l'architecture :
# 1) Reachability sieve -- test  $O(1)$  d'existence d'un terme, pas d'expansion
# 2) Certificat par exposants extremes -- non-annulation directe, sans  $R_a$ 
# 3) Table des  $R_a(d)$  via la recursion Gamma p-adique (Pochhammer)
# 4) Croisement formule vs extraction multinomiale directe, dans le meme run
#     (aucun etat partage entre les deux calculs : verification "double
#     implementation" au sens du Principe 4, meme au sein d'un seul script)

import time
from sage.combinat.permutation import Permutations as Perms

def reachable(D, d, n):
    """D (degre) atteignable dans  $(x^d + \sigma x + 1)^n$  ?  $O(1)$ ."""
    if D < 0 or D > d * n:
        return False
    lo = max(0, -((-D - n) // (d - 1))) if D > n else 0
    hi = min(n, D // d)
    return lo <= hi

def structural_exceptional_set(d, pmax):
    """Principe 1 : primes  $p < pmax$  pour lesquels une ligne/colonne de  $M_p$ 
    est structurellement vide (test  $O(1)$  par  $(i, j, p)$ )."""
    g = (d - 1) // 2
    hits = []
    for p in primes(3, pmax):
        n = (p - 1) // 2
        empty_row = False
        for i in range(1, g + 1):
            if all(not reachable(p * i - j, d, n) for j in range(1, g + 1)):
                empty_row = True
                break
        if empty_row:
            hits.append(p)
            continue
        for j in range(1, g + 1):
            if all(not reachable(p * i - j, d, n) for i in range(1, g + 1)):
                hits.append(p)
                break
    return hits

def ratio_d(d, beta, j):
    """Principe 3 : rapport  $\rho_{\{i, j+1\}} / \rho_{\{i, j\}}$  via la recursion
    Gamma p-adique / Pochhammer, independant de p."""
    alpha = QQ(-(j + beta)) / QQ(d)
    gamma = QQ(-1) / QQ(2) - alpha - beta
    if beta >= 1:
        return QQ(beta) / (gamma + 1)
    r = alpha
    for t in range(d - 2):
        r *= (gamma - t)
    fact = 1
    for k in range(2, d):
        fact *= k
```

```

return r / fact

def Ra_table(d):
    """Table des  $R_a^d$ ,  $a = p \bmod d$  ( $a=0$  exclu,  $\gcd(a,3)=1$  si  $3|d$ )."""
    g = (d - 1) // 2
    table = {}
    for a in range(1, d):
        if d % 3 == 0 and a % 3 == 0:
            table[a] = (None, None, QQ(0))
            continue
        beta = {(i, j): (a * i - j) % d
                 for i in range(1, g + 1) for j in range(1, g + 1)}
        rho = {}
        for i in range(1, g + 1):
            rho[(i, 1)] = QQ(1)
            for j in range(1, g):
                rho[(i, j + 1)] = rho[(i, j)] * ratio_d(d, beta[(i, j)], j)
        best = None
        T = []
        for pm in Perms(list(range(1, g + 1))):
            pm = tuple(pm)
            s = sum(beta[(i, pm[i - 1])] for i in range(1, g + 1))
            if best is None or s < best:
                best = s
                T = [pm]
            elif s == best:
                T.append(pm)

        def sgn(pm):
            inv = sum(1 for x in range(g) for y in range(x + 1, g)
                      if pm[x] > pm[y])
            return -1 if inv % 2 else 1

        tot = QQ(0)
        for pm in T:
            t = QQ(sgn(pm))
            for i in range(1, g + 1):
                t *= rho[(i, pm[i - 1])]
            tot += t
        table[a] = (best, T, tot)
    return table

def entries_low(d, p):
    """Terme de plus bas degre en sigma de chaque entree (i,j),  $1 \leq i, j \leq g$ ."""
    g = (d - 1) // 2
    n = (p - 1) // 2
    fact = [1] * (n + 1)
    for k in range(1, n + 1):
        fact[k] = fact[k - 1] * k % p
    inv = [pow(f, -1, p) for f in fact]
    low = {}
    for i in range(1, g + 1):
        for j in range(1, g + 1):
            N = p * i - j
            al = N // d
            be = N - d * al
            ga = n - al - be
            if ga < 0 or al > n:
                return None
            low[(i, j)] = (be, fact[n] * inv[al] % p * inv[be] % p * inv[ga] % p)
    return low

```

```

def fr_mod(x, p):
    return int(x.numerator()) % p * pow(int(x.denominator()) % p, -1, p) % p

def formula_check(d, table, pmin, pmax):
    """Principe 4 (au sein du run) : formule (prod m_i1)*R_a vs extraction
    multinomiale directe -- deux chemins de calcul independants."""
    g = (d - 1) // 2
    tested = 0
    bad = []
    for p in primes(pmin, pmax):
        a = p % d
        if a == 0 or table[a][2] == 0:
            continue
        low = entries_low(d, p)
        if low is None:
            continue
        best = None
        Tp = []
        for pm in Perms(list(range(1, g + 1))):
            pm = tuple(pm)
            s = sum(low[(i, pm[i - 1])][0] for i in range(1, g + 1))
            if best is None or s < best:
                best = s
                Tp = [pm]
            elif s == best:
                Tp.append(pm)

        def sgn(pm):
            inv = sum(1 for x in range(g) for y in range(x + 1, g)
                      if pm[x] > pm[y])
            return -1 if inv % 2 else 1

        tot = 0
        for pm in Tp:
            t = sgn(pm)
            for i in range(1, g + 1):
                t = t * low[(i, pm[i - 1])][1] % p
            tot = (tot + t) % p
        prod = 1
        for i in range(1, g + 1):
            prod = prod * low[(i, 1)][1] % p
        tested += 1
        if tot != prod * fr_mod(table[a][2], p) % p:
            bad.append(p)
    return tested, bad

def extremal_certificate(d, pmin, pmax):
    """Principe 2 : certificat de non-annulation par exposants extremes,
    sans jamais invoquer R_a."""
    g = (d - 1) // 2
    excl = set(structural_exceptional_set(d, pmin)) if pmin > 3 else set()
    ok_unique = 0
    ok_tie = 0
    incomplete = []
    fail = []
    for p in primes(max(pmin, 3), pmax):
        n = (p - 1) // 2
        E = {}
        incomplete_here = False
        for i in range(1, g + 1):
            for j in range(1, g + 1):
                N = p * i - j
                a_lo = max(0, -((-N - n)) // (d - 1)))

```

```

        a_hi = N // d
        if a_lo > a_hi:
            incomplete_here = True
            E[(i, j)] = (a_lo, a_hi)
    if incomplete_here:
        incomplete.append(p)
        continue
    vals = []
    for pm in Perms(list(range(1, g + 1))):
        pm = tuple(pm)
        s_min = sum(N - d * E[(i, pm[i - 1])][1]
                    for i, N in [(i, p * i - pm[i - 1]) for i in range(1, g + 1)])
        vals.append((s_min, pm))
    mn = min(v[0] for v in vals)
    Tmin = [pm for s, pm in vals if s == mn]
    if len(Tmin) == 1:
        ok_unique += 1
    else:
        ok_tie += 1 # tie set exists; a real coefficient sum would be needed
    return {"incomplete": incomplete, "ok_unique": ok_unique,
            "ok_tie_untested": ok_tie, "fail": fail}

if __name__ == "__main__":
    D = 7 # <-- changez ici pour tester un autre d impair (5, 9, 11, 13, ...)
    PMAX_STRUCT = 200000
    PMIN_FORM, PMAX_FORM = 101, 6000

    print("=" * 70)
    print(f"  ARCHITECTURE SOFYEN -- famille  $x^{\{D\}} + \sigma x + 1$  (genre  $\{(D-1)//2\})$ ")
    print("=" * 70)

    print(f"\n=== Principe 1 : ensemble exceptionnel structurel,  $p < \{PMAX\_STRUCT\}$  ===")
    t0 = time.time()
    exc = structural_exceptional_set(D, PMAX_STRUCT)
    print(" ", exc, f" ({time.time()-t0:.1f}s)")

    print(f"\n=== Principe 3 : table des  $R_a$  ( $a = p \bmod \{D\}$ ) ===")
    table = Ra_table(D)
    for a, (mn, T, val) in table.items():
        print(f"  a={a}: min_beta={mn}, |T|={len(T) if T else 0},  $R_a=\{val\}$ ")
    maxp = 2
    for a, (mn, T, val) in table.items():
        if val:
            for q in list(factor(abs(val.numerator())) + list(factor(val.denominator()))):
                maxp = max(maxp, q[0])
    print(f"  plus grand premier dans les  $R_a$  : {maxp} -> preuve algebrique pour tout  $p > \{maxp\}$ ")

    print(f"\n=== Principe 4 : formule vs extraction directe,  $\{PMIN\_FORM\} \leq p < \{PMAX\_FORM\}$  ===")
    tested, bad = formula_check(D, table, PMIN_FORM, PMAX_FORM)
    print(f"  premiers testes: {tested} | desaccords: {bad}")

    print("\nTermine.")

```